

A continuación presento el análisis completo de arquitectura para "El Último Bastión". Comenzaré con los diagramas estructurales y el flujo del sistema, seguidos del análisis técnico detallado.

El Último Bastión — Análisis y Diseño de Arquitectura de Software

1. Arquitectura General del Proyecto

El juego sigue una arquitectura en capas combinada con el patrón **Game Loop**, estructurada en tres niveles principales:

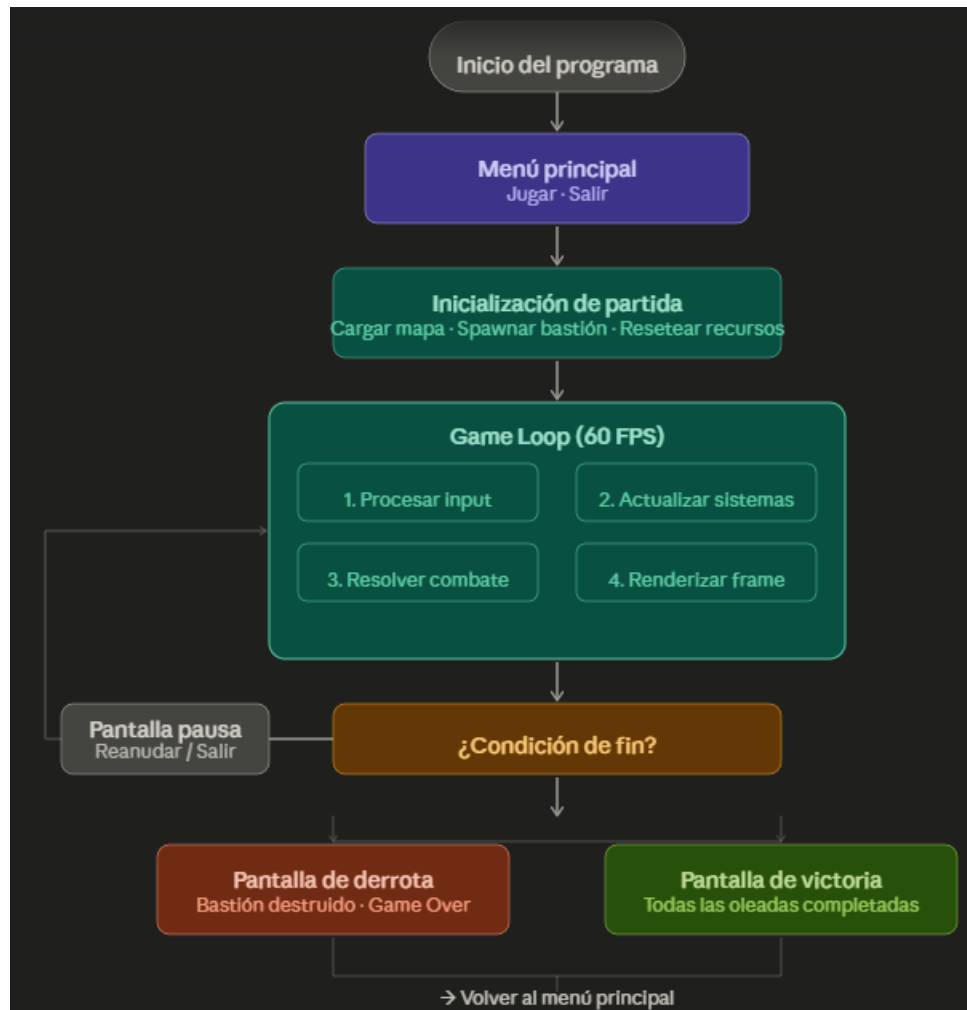
- **Capa de presentación:** Pygame se encarga del renderizado, inputs y audio.
- **Capa lógica:** Contiene todos los sistemas del juego (oleadas, combate, recursos, construcción).
- **Capa de datos:** Configuraciones, definiciones de entidades y estado de partida.

El patrón arquitectónico central es **Entity-Component-System (ECS) simplificado**, complementado con **State Machine** para el control de pantallas y **Observer/Event Bus** para la comunicación desacoplada entre sistemas.

El ciclo principal del juego opera de la siguiente manera: en cada frame se procesan los inputs del jugador, se actualiza el estado de todos los sistemas (oleadas, enemigos, torres, recursos), se resuelven colisiones y combate, y finalmente se renderiza el resultado. Este ciclo se ejecuta a 60 FPS mediante el Clock de Pygame.



2. Flujo del Videojuego



3. Estructura de Carpetas Recomendada

el_ultimo_bastion/

```

|
| └─ src/                # Código fuente principal
|   |
|   | └─ core/           # Núcleo del motor
|   |   |
|   |   | └─ game.py      # Game loop principal, inicialización
|   |   | └─ state_machine.py  # Máquina de estados (pantallas)
|   |   | └─ event_bus.py  # Sistema de eventos (Observer)
|   |   | └─ settings.py   # Constantes globales y config runtime
|   |   | └─ asset_loader.py # Carga centralizada de recursos
|   |   |
|   |   └─ entities/      # Entidades del juego

```

```
| | └─ base_entity.py    # Clase base abstracta
| | └─ enemies/
| |   └─ base_enemy.py
| |   └─ goblin_raider.py
| |   └─ goblin_archer.py
| |   └─ goblin_brute.py
| | └─ defenses/
| |   └─ base_defense.py
| |   └─ archer_tower.py
| |   └─ defensive_wall.py
| |   └─ ballista.py
| | └─ bastion.py        # El bastión (entidad especial)
| |
| └─ systems/           # Sistemas de lógica
|   └─ wave_system.py   # Control de oleadas
|   └─ combat_system.py # Resolución de combate / colisiones
|   └─ resource_system.py # Gestión de oro y maná
|   └─ build_system.py  # Construcción y colocación
|   └─ pathfinding.py   # Navegación de enemigos (A*)
|   └─ map_system.py    # Grid del mapa, celdas válidas
|   |
|   └─ screens/         # Pantallas (estados)
|     └─ main_menu.py
|     └─ gameplay.py
|     └─ pause_screen.py
|     └─ defeat_screen.py
|     |
|   └─ ui/              # Interfaz de usuario
|     └─ hud.py         # HUD en gameplay (recursos, vida, oleada)
```

```
| | └─ build_panel.py    # Panel de construcción
| | └─ tooltip.py       # Tooltips de estructuras
| | └─ button.py        # Componente botón reutilizable
| |
| └─ utils/             # Utilidades transversales
|   └─ timer.py         # Temporizadores reutilizables
|   └─ animation.py     # Gestión de sprites animados
|   └─ math_utils.py    # Distancias, vectores 2D
|
└─ assets/              # Recursos estáticos
    └─ sprites/
        └─ enemies/
        └─ towers/
        └─ ui/
        └─ map/
        └─ effects/
    └─ sounds/
        └─ sfx/
        └─ music/
    └─ fonts/
|
└─ data/                # Configuración en datos (no código)
    └─ enemies.json     # Stats de todos los enemigos
    └─ towers.json      # Costos y stats de torres
    └─ waves.json       # Definición de cada oleada
    └─ maps/
        └─ map_01.json  # Layout del mapa
|
└─ tests/               # Pruebas unitarias
```

```

|   |— test_wave_system.py
|   |— test_combat_system.py
|   |— test_resource_system.py
|   |— test_pathfinding.py
|
|— docs/                # Documentación
|   |— architecture.md
|   |— game_design.md
|   |— api_reference.md
|
|— main.py              # Punto de entrada
|— requirements.txt
|— README.md

```

La separación entre `data/` y `src/` es una decisión crítica: permite que el balanceo del juego (vida de enemigos, costos de torres, composición de oleadas) se ajuste sin tocar código fuente, facilitando la iteración de diseño y las pruebas.

4. Módulos del Sistema

Módulo	Responsabilidad principal	Dependencias
<code>core/game.py</code>	Inicializar Pygame, ejecutar el game loop, delegar a la state machine	pygame, state_machine
<code>core/state_machine.py</code>	Gestionar qué pantalla está activa, transiciones entre estados	screens/*
<code>core/event_bus.py</code>	Publicar y suscribir eventos globales (patrón Observer)	Ninguna (capa base)
<code>core/asset_loader.py</code>	Cargar sprites, sonidos y fuentes; mantener caché	pygame, data/

Módulo	Responsabilidad principal	Dependencias
systems/wave_system.py	Leer waves.json, generar enemigos en tiempo y posición, escalar dificultad	event_bus, entities/enemies
systems/combat_system.py	Detectar rango de torres, aplicar daño, gestionar proyectiles	entities/*, event_bus
systems/resource_system.py	Mantener saldo de oro/maná, procesar ingresos y gastos	event_bus
systems/build_system.py	Validar celdas libres, descontar recursos, instanciar estructuras	map_system, resource_system
systems/pathfinding.py	Calcular rutas A* sobre el grid para los enemigos	map_system
systems/map_system.py	Representar el grid de celdas, marcar ocupadas/libres/camino	Ninguna
entities/base_entity.py	Clase abstracta con posición, update(), draw(), estado activo	pygame
entities/enemies/base_enemy.py	Movimiento por path, recibir daño, morir, recompensar oro	pathfinding, event_bus
entities/defenses/base_defense.py	Detección de enemigos en rango, cooldown de ataque, disparar	combat_system, event_bus
ui/hud.py	Renderizar vida del bastión, recursos actuales, número de oleada	resource_system, bastion
ui/build_panel.py	Mostrar estructuras disponibles, costos, selección activa	resource_system

5. Diseño de Clases Principales

A continuación se detallan las responsabilidades y relaciones de las clases más importantes:

BaseEntity — Clase abstracta raíz. Define la interfaz común de todas las entidades del juego: posición, estado activo, y los métodos update(dt) y draw(surface). El uso del parámetro dt (delta time) en lugar de un valor fijo garantiza que el juego corra a la misma velocidad independientemente del framerate.

BaseEnemy — Extiende BaseEntity. Agrega los atributos de combate (HP, velocidad, daño) y la lógica de movimiento por pathfinding. Al morir, emite un evento `enemy_died` al EventBus en lugar de llamar directamente al sistema de recursos, lo que mantiene el bajo acoplamiento.

BaseDefense — Extiende BaseEntity. Abstrae el comportamiento de todas las defensas: detección de enemigos en rango, cooldown de ataque, y descuento de recursos al construir. `DefensiveWall` sobrescribe `attack()` con una implementación vacía.

WaveSystem — Lee las definiciones desde `waves.json` y gestiona el ritmo de la partida: cuándo termina una oleada, cuándo empieza la siguiente, y el escalado de dificultad. Incrementa la dificultad multiplicando el HP y la cantidad de enemigos por un factor configurable por oleada.

EventBus — Clase singleton que implementa el patrón Observer. Los sistemas se suscriben a eventos por nombre (`bus.subscribe("enemy_died", callback)`) y los publican sin conocer quién los escucha (`bus.publish("enemy_died", data)`). Esto es lo que permite agregar nuevos sistemas (por ejemplo, un sistema de estadísticas) sin modificar el código existente.

6. Escalabilidad de la Arquitectura

La arquitectura está diseñada para crecer sin necesidad de refactorizaciones grandes:

Nuevos enemigos: Crear una clase que herede `BaseEnemy`, sobrescribir los atributos de stats, y agregar la entrada al archivo `enemies.json`. No se toca ningún sistema existente. Ejemplo: `GoblinShaman` con la capacidad de curar a aliados sobrescribiendo el método `update()`.

Nuevas torres: Idéntico al proceso anterior con `BaseDefense`. La `ArcherTower` con ataque de área sobrescribiría `attack()` para aplicar daño en radio en lugar de objetivo único.

Nuevos mapas: Agregar un archivo JSON en `data/maps/`. El `MapSystem` los consume sin cambios en código. Los paths de enemigos se definen en el JSON, no en el código.

Nuevos modos de juego: La `StateMachine` admite nuevos estados. Se crea una nueva pantalla (por ejemplo, `endless_mode.py`), se registra en la máquina de estados, y se agrega la opción al menú. El resto del motor no se modifica.

Sistema de mejoras de torres: Se agrega un atributo `level` a `BaseDefense` y un método `upgrade()`. El `BuildSystem` expone esta opción en el `BuildPanel`. Ninguna clase existente necesita cambios estructurales.

Guardado de partida: El EventBus ya separa el estado. Se agrega un módulo `save_system.py` que serializa el estado de todos los sistemas a JSON en respuesta al evento `game_paused`.

7. Riesgos Técnicos y Mitigaciones

Riesgo	Impacto	Mitigación
Caídas de FPS con muchos enemigos	Alto — rompe la jugabilidad	Implementar object pooling para enemigos y proyectiles; evitar new/del en cada frame. Usar <code>pygame.sprite.Group</code> con <code>LayeredUpdates</code>
Pathfinding costoso en tiempo real	Medio — lag perceptible	Precalcular paths al inicio de la oleada, no en cada frame. Recalcular solo cuando el mapa cambia (nueva construcción)
Acoplamiento creciente entre sistemas	Alto — dificulta mantenimiento	Respetar el <code>EventBus</code> desde el inicio; nunca importar directamente entre sistemas de la misma capa
Colisiones imprecisas	Medio — frustración del jugador	Usar <code>pygame.sprite.spritecollide()</code> con <code>collide_circle</code> para proyectiles; las hitboxes deben ser ligeramente menores que el sprite
Escalado de dificultad mal calibrado	Medio — juego imposible o trivial	Definir dificultad en <code>waves.json</code> con multiplicadores; nunca hardcodear en las clases. Facilita el playtesting sin tocar código
Gestión de memoria con assets	Bajo/Medio — crashes en builds finales	Centralizar toda carga en <code>AssetLoader</code> con un diccionario de caché; cargar en startup, no en runtime
Delta time inconsistente	Bajo	Capear dt con <code>min(dt, 0.05)</code> para evitar que pausas del sistema operativo causen saltos de posición enormes

8. Recomendaciones Profesionales

Adoptar delta time desde el primer día. Todo movimiento debe multiplicarse por dt (tiempo transcurrido en segundos). Jamás usar `pygame.time.Clock.tick()` como movimiento directo. Esto garantiza que el juego se comporte igual a 30, 60 o 120 FPS.

El EventBus es no-negociable. La tentación en prototipos es hacer `from resource_system import resource_system` directamente desde el enemigo. Esto crea dependencias circulares que son muy costosas de deshacer. El `EventBus` debe implementarse en la primera sesión de desarrollo.

Separar configuración de código desde el inicio. Todos los valores numéricos de balance —vida de enemigos, costos de torres, composición de oleadas— deben vivir en

los archivos JSON desde el primer commit. Esto permite ajustar el juego sin riesgos de introducir bugs.

Usar pygame.sprite.Group y sus variantes. Pygame ya provee estructuras optimizadas para grupos de entidades. Usar listas Python simples y llamar draw()/update() manualmente en un loop es menos eficiente y más propenso a errores.

Implementar un sistema de logging desde el inicio. El módulo logging de Python es suficiente. Registrar eventos importantes (oleada iniciada, enemigo muerto, recurso gastado) facilita enormemente el debugging sin usar print().

Control de versiones por features. Usar ramas de Git por funcionalidad (feature/wave-system, feature/pathfinding). Nunca desarrollar directamente en main. Esto permite comparar el avance y revertir features rotas.

Pruebas unitarias para los sistemas, no para el render. Los sistemas (WaveSystem, ResourceSystem, CombatSystem) son lógica pura y completamente testeable sin Pygame. Las pruebas de render son costosas y frágiles. Priorizar tests de lógica de negocio.

Orden de implementación recomendado:

1. EventBus + StateMachine + Game Loop vacío
2. MapSystem con grid renderizable
3. BaseEntity + BaseEnemy + pathfinding básico
4. WaveSystem con spawning
5. BaseDefense + CombatSystem
6. ResourceSystem + BuildSystem
7. HUD + pantallas de menú/derrota
8. Arte y audio (nunca al principio)

Este orden garantiza que el núcleo jugable exista temprano, permitiendo iteraciones de diseño desde la primera semana.